Compiler Construction

(BCO 028B)

UNIT 1

Prepared By:

Ms. Priyanka Goyal

Department of Computer Science & Engineering
JECRC University

BCO 028B	COMPILER CONSTRUCTION	3-1-0 [4]

UNIT 1	Overview of compilation- The structure of a compiler and applications of compiler technology; Lexical analysis - The role of a lexical analyzer, specification of tokens, recognition of tokens, hand-written lexical analyzers, LEX, examples of LEX programs. Introduction to syntax analysis -Role of a parser, use of context-free grammars (CFG) in the specification of the syntax of programming languages, techniques for writing grammars for programming languages (removal left recursion, etc.), non-context-free constructs in programming languages, parse trees and ambiguity, examples of programming language grammars.

Course Outcome	Program Outcome												Program Specific Outcome		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	H	H	L		H			L					H	M	
CO2		H				L							H		
CO3		L		H	L										M
CO4		H					H							H	
CO5		H		L		H									L

Books & References

Text Books:

1. Compilers: Principles, Techniques, and Tools, by A.V. Aho, Monica Lam, Ravi Sethi, and J.D. Ullman, (2nded.), Addison-Wesley, 2007 (main text book, referred to as ALSU in lab assignments).
2. K.D. Cooper, and Linda Torczon, Engineering a Compiler, Morgan Kaufmann, 2004.

Reference Books:

1. K.C. Louden, Compiler Construction: Principles and Practice, Cengage Learning, 1997.
2. 2. D. Brown, J. Levine, and T. Mason, LEX and YACC, O‟Reilly Media, 1992.

Overview of Compilation

- Compilation is the process of converting a high-level programming language into machine code.
- It allows programs written by humans to run on computers.
- Example: C program → compiled → executable file (.exe)

Structure of a Compiler

A compiler works in phases:

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. Intermediate Code Generation
5. Code Optimization
6. Code Generation

Applications of Compiler Technology

- Programming language implementation (C, Java, Python)
- Database query optimization (SQL compilers)
- Hardware design (VHDL/Verilog compilers)
- Natural language processing tools
- AI
- Gaming
- Embedded Systems

Lexical Analyzer

Lexical Analysis – Role

- Lexical Analyzer (Scanner) reads source code and converts it into tokens.
- Removes whitespace and comments.
  Identification And categorization of tokens
- Example: int x = 10; → (KEYWORD,int) (ID,x) (=) (NUM,10) (;)

Specification of Tokens

- Tokens are basic language units.
- Types: Keywords, Identifiers, Operators, Literals, Separators, Special Symbols.
- Example: 'while', 'count', '+', '25', ';'

Recognition of Tokens

- Tokens are recognized using patterns (regular expressions).
- Example patterns:
- Identifier → letter(letter|digit)*
- Number → digit+

Hand-Written Lexical Analyzer

- Programmer manually writes code to identify tokens.
- Uses loops and condition checks.
- Example: Checking characters one by one in C using getchar().

LEX Tool

- LEX is a tool to automatically generate lexical analyzers.
- Input: Token rules using regular expressions.
- Output: C program that performs lexical analysis.

Example of a LEX Program

- %%
- [0-9]+ { printf("NUMBER"); }
- [a-zA-Z]+ { printf("WORD"); }
- %%
- This program identifies numbers and words.

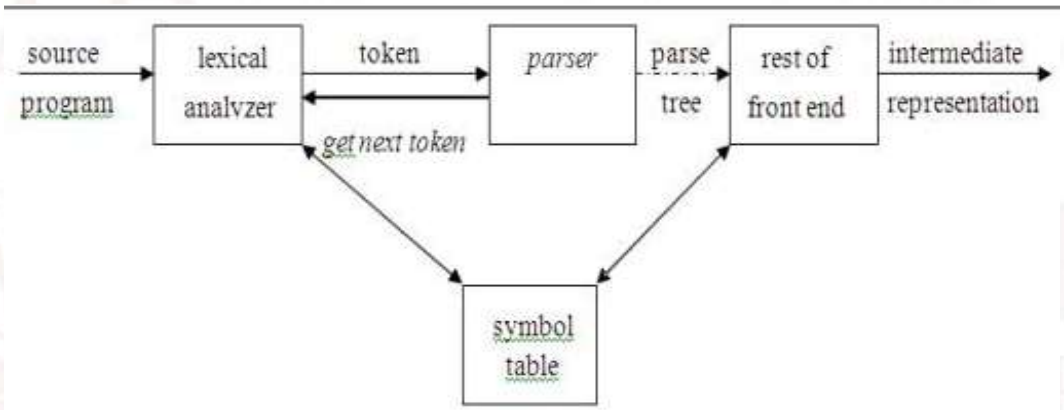
Syntax Analyzer

Introduction to Syntax Analysis

- Syntax Analysis (Parsing) checks whether token sequence follows grammar rules.
- Builds a parse tree.
- Example: Verifying if 'a + b' follows expression grammar.

Role of a Parser

- Parser takes tokens from lexical analyzer.
- Detects syntax errors.
- Creates parse tree or syntax tree.
- It constructs the parse tree.
- It performs error recovery.



Context-Free Grammars (CFG)

- CFG is used to describe programming language syntax.
- Consists of terminals, non-terminals, productions, start symbol.
- Example: $E \rightarrow E + T \mid T$

Grammar Writing Techniques

- Removing Left Recursion:
- $E \rightarrow E + T \mid T$ becomes $E \rightarrow T\ E'$
- $E' \rightarrow + T\ E' \mid \varepsilon$
- Left Factoring improves parser efficiency.

Non-Context-Free Constructs

- Some language rules cannot be described by CFG alone.
- Example: Variable declaration before use.
- Handled during semantic analysis.

Parse Tree

- Graphical representation of derivation of a string
- Root → Start symbol
- Internal nodes → Non-terminals
- Leaf nodes → Terminals
- Shows syntactic structure clearly

Grammar

- Grammar G = (V, T, P, S)

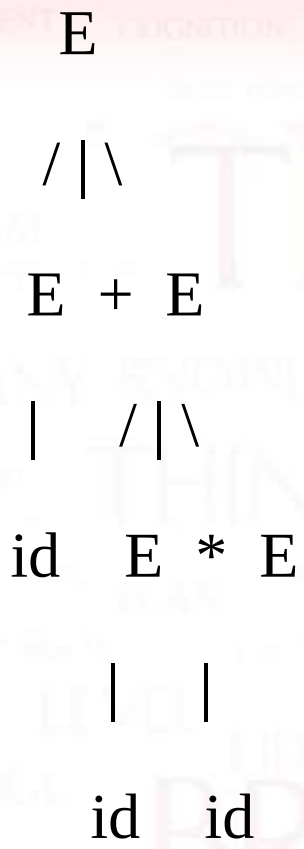
- V → Non-terminals
- T → Terminals
- P → Production rules
- S → Start symbol.

Example Grammar/ Example String

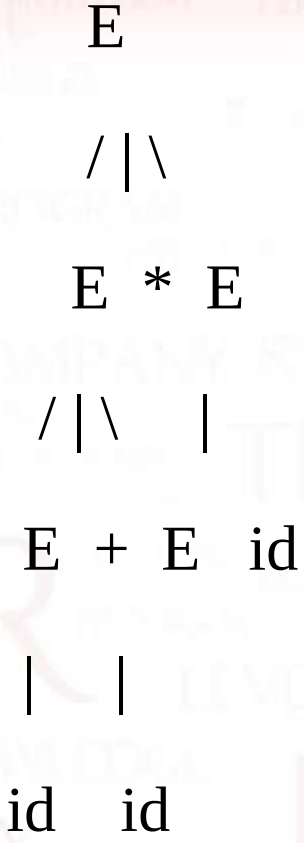
- $E \rightarrow E + E$
- $E \rightarrow E * E$
- $E \rightarrow id$
- This grammar is ambiguous

- String: id + id * id
- This string can have more than one parse tree.

Parse Tree 1: + Evaluated First



Parse Tree 2: * Evaluated First



Parse Trees and Ambiguity

- Parse tree shows grammatical structure.
- Ambiguous grammar: One string → multiple parse trees.
- Example: E → E + E | E * E | id

Example Programming Language Grammar

- stmt → if expr then stmt else stmt | while expr do stmt | id = expr
- expr → expr + term | term
- term → id | number

Thank You